

Pack 15 (Desktop) — Device Access Policy Enforcement + Remote Input Safety Gates (real code)

RemoteDesk · Pack 15 (Desktop) — Device Access Policy Enforcement + Remote Input Safety Gates (real code)

Codex / Claude Code handoff. Desktop-focused slice. Copy each file to the exact path in its `FILE:` heading. Build order: shared contract → desktop policy client + evaluator → session enforcement → input gates → emergency stop → UI → tests.

Scope: enforce persisted device access policies inside the desktop session flow. **Fail-closed everywhere.** **Remote input disabled by default. Host is the source of truth for permissions. Native OS input execution stays disabled/no-op (no new native executor).** **No secrets in logs.**

Tag legend: **[NEW]** = create file (safe direct copy). **[MODIFY]** = patch an existing file by hand (merge into current contents, do not overwrite wholesale).

Files created / modified / not touched

CREATED [NEW]	
packages/shared/src/permissions/devicePolicy.ts	
packages/shared/src/permissions/policyEvaluator.ts	
packages/shared/src/permissions/index.ts	
apps/desktop/src/renderer/src/services/devicePolicyClient.ts	
apps/desktop/src/renderer/src/services/permissionState.ts	
apps/desktop/src/renderer/src/services/remoteInputGate.ts	
apps/desktop/src/renderer/src/services/emergencyStop.ts	
apps/desktop/src/renderer/src/components/PolicyStatusBanner.tsx	
apps/desktop/src/renderer/src/components/EmergencyStopButton.tsx	
tests/permissions/policyEvaluator.test.ts	
MODIFIED [MODIFY]	
packages/shared/src/index.ts	(export permissions barrel)
apps/desktop/src/renderer/src/services/sessionDataChannel.ts	(permission snapshot frames)
apps/desktop/src/renderer/src/services/webrtc.ts	(gate input send/recv on policy)
apps/desktop/src/renderer/src/components/RemoteSessionView.tsx	(banner + emergency stop + read-only)
NOT TOUCHED (intentionally)	
apps/desktop/src/main/index.ts	(no native input executor added)
apps/desktop/src/preload/index.ts	(no new privileged bridge needed)
apps/desktop/src/renderer/src/services/socket.ts	(signaling unchanged)
apps/api/** apps/web/**	(policy already persisted server-side)

Part A — Shared policy contract

FILE: packages/shared/src/permissions/devicePolicy.ts

```

/**
 * Shared device access policy contract. The desktop enforces these; the API/web
 * already persist them. Everything here is plain data + string unions so it can
 * cross the data channel and be reasoned about in pure functions.
 */

export type DeviceTrustStatus = 'trusted' | 'untrusted' | 'blocked';

/** A point-in-time snapshot of a device's persisted access policy. */
export interface DeviceAccessPolicySnapshot {
  deviceId: string;
  trustStatus: DeviceTrustStatus;
  unattendedAccessEnabled: boolean;
  remoteInputEnabled: boolean;
  clipboardSyncEnabled: boolean;
  fileTransferEnabled: boolean;
  requiresSessionApproval: boolean;
  /** Hard cap in minutes; null = no cap. */
  maxSessionMinutes: number | null;
  /** ISO timestamp the policy was last updated server-side. */
  updatedAt: string;
}

/**
 * The effective, runtime permission set for a live session. Derived by the host
 * from the policy + host acceptance + emergency-stop state, then broadcast to the
 * viewer over the data channel. The HOST is the source of truth.
 */
export interface SessionPermissionSet {
  deviceId: string;
  remoteInput: RemoteInputPermissionState;
  clipboardSync: boolean;
  fileTransfer: boolean;
  /** ISO expiry derived from maxSessionMinutes, or null if uncapped. */
  sessionExpiresAt: string | null;
  /** True while the host emergency stop is engaged. */
  emergencyStopped: boolean;
  /** Monotonic version so the viewer ignores stale frames. */
  version: number;
  updatedAt: string;
}

/** Remote input is a tri-state so the UI can distinguish "off" from "not allowed". */
export type RemoteInputPermissionState = 'enabled' | 'disabled' | 'blocked';

/** Why a policy decision denied something. Drives user-facing messaging. */
export type PolicyDenialReason =
  | 'device_blocked'
  | 'device_untrusted'
  | 'missing_policy'
  | 'feature_disabled_by_policy'
  | 'host_not_accepted'
  | 'emergency_stop_active'
  | 'session_expired';

/** Result of a pure policy evaluation. */

```

```

export interface PolicyDecision {
  allowed: boolean;
  /** Set only when allowed === false. */
  reason: PolicyDenialReason | null;
}

/** Convenience constructors keep call sites terse + consistent. */
export function allow(): PolicyDecision {
  return { allowed: true, reason: null };
}
export function deny(reason: PolicyDenialReason): PolicyDecision {
  return { allowed: false, reason };
}

```

FILE: packages/shared/src/permissions/policyEvaluator.ts

```

import {
  allow,
  deny,
  type DeviceAccessPolicySnapshot,
  type PolicyDecision,
  type PolicyDenialReason,
  type RemoteInputPermissionState,
  type SessionPermissionSet,
} from './devicePolicy';

/**
 * Pure, fail-closed policy evaluation shared by the host (enforcement) and the
 * UI (gating). No I/O, no Date.now() side effects beyond explicit expiry math.
 *
 * Core rules:
 * - blocked device          => deny everything
 * - untrusted device        => deny unattended access + remote input
 * - missing policy (null)   => deny all risky features
 * - remote input is OFF by default; host must accept first
 * - host acceptance required for remote input / clipboard / file transfer
 */

/** Gate session start itself. A blocked device cannot even start a session. */
export function evaluateSessionStartPolicy(
  policy: DeviceAccessPolicySnapshot | null,
): PolicyDecision {
  if (!policy) return deny('missing_policy');
  if (policy.trustStatus === 'blocked') return deny('device_blocked');
  return allow();
}

/** Can remote input be enabled right now? */
export function canEnableRemoteInput(
  policy: DeviceAccessPolicySnapshot | null,
  hostAccepted: boolean,
  emergencyStopped = false,
): PolicyDecision {
  if (!policy) return deny('missing_policy');
  if (policy.trustStatus === 'blocked') return deny('device_blocked');
  if (policy.trustStatus === 'untrusted') return deny('device_untrusted');
}

```

```

    if (!policy.remoteInputEnabled) return deny('feature_disabled_by_policy');
    if (emergencyStopped) return deny('emergency_stop_active');
    if (!hostAccepted) return deny('host_not_accepted');
    return allow();
}

/** Can clipboard sync be enabled? (Untrusted is allowed clipboard, not input.) */
export function canEnableClipboard(
    policy: DeviceAccessPolicySnapshot | null,
    hostAccepted: boolean,
): PolicyDecision {
    if (!policy) return deny('missing_policy');
    if (policy.trustStatus === 'blocked') return deny('device_blocked');
    if (!policy.clipboardSyncEnabled) return deny('feature_disabled_by_policy');
    if (!hostAccepted) return deny('host_not_accepted');
    return allow();
}

/** Can file transfer be enabled? */
export function canEnableFileTransfer(
    policy: DeviceAccessPolicySnapshot | null,
    hostAccepted: boolean,
): PolicyDecision {
    if (!policy) return deny('missing_policy');
    if (policy.trustStatus === 'blocked') return deny('device_blocked');
    if (!policy.fileTransferEnabled) return deny('feature_disabled_by_policy');
    if (!hostAccepted) return deny('host_not_accepted');
    return allow();
}

/** Human-readable message for a denial reason. UI-facing, non-sensitive. */
export function getPolicyDenialMessage(reason: PolicyDenialReason): string {
    switch (reason) {
        case 'device_blocked':
            return 'This device is blocked. No remote features are available.';
        case 'device_untrusted':
            return 'This device is untrusted. Remote input and unattended access are disabled.';
        case 'missing_policy':
            return 'Device policy could not be loaded. Risky features are disabled for safety.';
        case 'feature_disabled_by_policy':
            return 'This feature is disabled by the device access policy.';
        case 'host_not_accepted':
            return 'Waiting for the host to allow this feature.';
        case 'emergency_stop_active':
            return 'Emergency stop is active. Remote input is disabled.';
        case 'session_expired':
            return 'The session time limit has been reached.';
        default:
            return 'This action is not allowed by policy.';
    }
}

/** Derive a session expiry ISO timestamp from maxSessionMinutes. */
export function deriveSessionExpiry(
    policy: DeviceAccessPolicySnapshot | null,
    startedAt: Date = new Date(),
): string | null {

```

```

    if (!policy || policy.maxSessionMinutes === null) return null;
    return new Date(startedAt.getTime() + policy.maxSessionMinutes * 60_000).toISOString();
}

/** Is a session past its derived expiry? */
export function isSessionExpired(
  sessionExpiresAt: string | null,
  now: Date = new Date(),
): boolean {
  if (!sessionExpiresAt) return false;
  return now.getTime() >= Date.parse(sessionExpiresAt);
}

/**
 * Build the effective permission set the host broadcasts to the viewer. This is
 * the single place runtime permissions are composed, so host + viewer never drift.
 */
export function buildSessionPermissionSet(input: {
  policy: DeviceAccessPolicySnapshot | null;
  hostAccepted: boolean;
  emergencyStopped: boolean;
  startedAt?: Date;
  version: number;
}): SessionPermissionSet {
  const { policy, hostAccepted, emergencyStopped, version } = input;
  const deviceId = policy?.deviceId ?? 'unknown';
  const expires = deriveSessionExpiry(policy, input.startedAt ?? new Date());

  const inputDecision = canEnableRemoteInput(policy, hostAccepted, emergencyStopped);
  let remoteInput: RemoteInputPermissionState;
  if (inputDecision.allowed) {
    remoteInput = 'enabled';
  } else if (
    inputDecision.reason === 'device_blocked' ||
    inputDecision.reason === 'device_untrusted'
  ) {
    remoteInput = 'blocked';
  } else {
    remoteInput = 'disabled';
  }

  return {
    deviceId,
    remoteInput,
    clipboardSync: canEnableClipboard(policy, hostAccepted).allowed,
    fileTransfer: canEnableFileTransfer(policy, hostAccepted).allowed,
    sessionExpiresAt: expires,
    emergencyStopped,
    version,
    updatedAt: new Date().toISOString(),
  };
}

```

FILE: packages/shared/src/permissions/index.ts

```
export * from './devicePolicy';
export * from './policyEvaluator';
```

FILE: packages/shared/src/index.ts [MODIFY]

```
// ...existing exports...
export * from './permissions';
```

Part B — Desktop policy client

FILE: apps/desktop/src/renderer/src/services/devicePolicyClient.ts

```
import {
  evaluateSessionStartPolicy,
  type DeviceAccessPolicySnapshot,
} from '@remotedesk/shared';

/**
 * Fetches the persisted device access policy from the API. If the exact route
 * differs in your API, change ONLY this constant — everything else is decoupled.
 */
export const DEVICE_POLICY_ENDPOINT = (deviceId: string): string =>
  `/api/devices/${encodeURIComponent(deviceId)}/policy`;

export interface PolicyFetchResult {
  /** null when the policy could not be loaded (caller MUST fail closed). */
  policy: DeviceAccessPolicySnapshot | null;
  /** True if served from the display-only cache (not safe to enable features). */
  fromCache: boolean;
}

/**
 * Policy client. On any failure it returns `policy: null` so the evaluator fails
 * closed. The last known policy is cached ONLY for display (e.g. showing the last
 * trust status) and is never returned as the authoritative policy for enabling
 * risky features. No secrets are logged.
 */
export class DevicePolicyClient {
  private displayCache: DeviceAccessPolicySnapshot | null = null;

  constructor(
    private readonly baseUrl: string,
    private readonly authToken: string,
  ) {}

  /** Authoritative fetch. Returns null policy on any error (fail closed). */
  async fetch(deviceId: string): Promise<PolicyFetchResult> {
    try {
      const res = await fetch(`${this.baseUrl}${DEVICE_POLICY_ENDPOINT(deviceId)}`, {
        headers: { authorization: `Bearer ${this.authToken}` },
      });
      if (!res.ok) throw new Error(`policy fetch status ${res.status}`);
      const policy = (await res.json()) as DeviceAccessPolicySnapshot;
      // Sanity: enforce the start gate so a blocked device is obvious to callers.
    }
  }
}
```

```

        evaluateSessionStartPolicy(policy);
        this.displayCache = policy;
        return { policy, fromCache: false };
    } catch (err) {
        // Non-sensitive structured diagnostic only.
        console.warn('[policy] fetch failed, failing closed', { deviceId, reason: errMessage(err) });
        return { policy: null, fromCache: false };
    }
}

/** Display-only last known policy. NEVER use this to enable risky features. */
lastKnownForDisplay(): DeviceAccessPolicySnapshot | null {
    return this.displayCache;
}
}

function errMessage(err: unknown): string {
    return err instanceof Error ? err.message : 'unknown';
}

```

FILE: apps/desktop/src/renderer/src/services/permissionState.ts

```

import {
    buildSessionPermissionSet,
    type DeviceAccessPolicySnapshot,
    type SessionPermissionSet,
} from '@remotedesk/shared';

/**
 * Host-side authoritative permission state. Holds the policy + host-acceptance +
 * emergency-stop flags and produces a versioned SessionPermissionSet to broadcast.
 * The viewer only ever CONSUMES a permission set; it never computes its own.
 */
export class HostPermissionState {
    private policy: DeviceAccessPolicySnapshot | null = null;
    private hostAccepted = false;
    private emergencyStopped = false;
    private version = 0;
    private startedAt = new Date();
    private listeners = new Set<(p: SessionPermissionSet) => void>();

    setPolicy(policy: DeviceAccessPolicySnapshot | null): void {
        this.policy = policy;
        this.emit();
    }

    /** Host explicitly accepts/relaxes risky features (e.g. clicks "Allow control"). */
    setHostAccepted(accepted: boolean): void {
        this.hostAccepted = accepted;
        this.emit();
    }

    setEmergencyStopped(stopped: boolean): void {
        this.emergencyStopped = stopped;
        this.emit();
    }
}

```

```

markStarted(at: Date = new Date()): void {
    this.startedAt = at;
}

current(): SessionPermissionSet {
    return buildSessionPermissionSet({
        policy: this.policy,
        hostAccepted: this.hostAccepted,
        emergencyStopped: this.emergencyStopped,
        startedAt: this.startedAt,
        version: this.version,
    });
}

subscribe(fn: (p: SessionPermissionSet) => void): () => void {
    this.listeners.add(fn);
    return () => this.listeners.delete(fn);
}

private emit(): void {
    this.version += 1;
    const snapshot = this.current();
    for (const fn of this.listeners) fn(snapshot);
}
}

```

Part C — Data channel permission frames

FILE: apps/desktop/src/renderer/src/services/sessionDataChannel.ts [MODIFY]

```

// ...existing imports...
import type { SessionPermissionSet } from '@remotedesk/shared';

// ---- ADD: a discriminated frame type for permission snapshots ----
// Merge this into your existing DataChannelFrame union.
export interface PermissionSnapshotFrame {
    type: 'permission.snapshot';
    payload: SessionPermissionSet;
}

// Example union extension (adapt to your existing union name):
// export type DataChannelFrame =
//   | ChatFrame | ClipboardFrame | FileTransferFrame | InputFrame
//   | PermissionSnapshotFrame;

// ---- ADD: helpers on your existing SessionDataChannel class/instance ----

/** HOST: broadcast the authoritative permission set to the viewer. */
export function sendPermissionSnapshot(
    send: (frame: PermissionSnapshotFrame) => void,
    permissions: SessionPermissionSet,
): void {
    send({ type: 'permission.snapshot', payload: permissions });
}

```

```

/**
 * VIEWER: handle an incoming permission snapshot. Ignores stale versions so an
 * out-of-order frame can't re-enable input after a revoke.
 */
export function makePermissionSnapshotHandler(
  onApply: (p: SessionPermissionSet) => void,
) {
  let lastVersion = -1;
  return (frame: PermissionSnapshotFrame): void => {
    const p = frame.payload;
    if (p.version <= lastVersion) return;
    lastVersion = p.version;
    onApply(p);
  };
}

```

Part D — Remote input safety gate

FILE: apps/desktop/src/renderer/src/services/remoteInputGate.ts

```

import type { SessionPermissionSet } from '@remotedesk/shared';

/**
 * Bidirectional remote-input gate.
 * - VIEWER side: only captures + sends input when permission is 'enabled'.
 * - HOST side: rejects any inbound input unless permission is 'enabled'.
 * Blocked input is logged as a structured, NON-SENSITIVE diagnostic (no key codes,
 * no coordinates beyond a count).
 */
export class RemoteInputGate {
  private permissions: SessionPermissionSet | null = null;
  private blockedCount = 0;

  update(permissions: SessionPermissionSet | null): void {
    this.permissions = permissions;
  }

  /** True only when input is explicitly enabled and not emergency-stopped/expired. */
  inputAllowed(now: Date = new Date()): boolean {
    const p = this.permissions;
    if (!p) return false; // fail closed
    if (p.emergencyStopped) return false;
    if (p.remoteInput !== 'enabled') return false;
    if (p.sessionExpiresAt && now.getTime() >= Date.parse(p.sessionExpiresAt)) return false;
    return true;
  }

  /** VIEWER: call before sending each input event. Returns false to drop it. */
  guardOutbound(): boolean {
    if (this.inputAllowed()) return true;
    this.recordBlocked('outbound');
    return false;
  }
}

```

```

/** HOST: call before executing each inbound input event. Returns false to reject. */
guardInbound(): boolean {
  if (this.inputAllowed()) return true;
  this.recordBlocked('inbound');
  return false;
}

blockedTotal(): number {
  return this.blockedCount;
}

private recordBlocked(direction: 'inbound' | 'outbound'): void {
  this.blockedCount += 1;
  // Structured + non-sensitive: no event payload, just a counter + reason.
  if (this.blockedCount === 1 || this.blockedCount % 50 === 0) {
    console.info('[input-gate] blocked input', {
      direction,
      state: this.permissions?.remoteInput ?? 'none',
      emergencyStopped: this.permissions?.emergencyStopped ?? false,
      blockedTotal: this.blockedCount,
    });
  }
}

```

FILE: apps/desktop/src/renderer/src/services/webrtc.ts [MODIFY]

```

// ...existing imports...
import { RemoteInputGate } from './remoteInputGate';
import type { SessionPermissionSet } from '@remotedesk/shared';

// ---- ADD: a shared gate instance for this session ----
export const remoteInputGate = new RemoteInputGate();

/** Call whenever a new permission snapshot is applied (host or viewer). */
export function applyPermissionsToWebRtc(permissions: SessionPermissionSet | null): void {
  remoteInputGate.update(permissions);
}

// ---- MODIFY: viewer-side input send path ----
// Find your existing function that sends mouse/keyboard frames over the input
// data channel and gate it. Example wrapper:
//
// export function sendInputEvent(event: InputEvent): void {
//   if (!remoteInputGate.guardOutbound()) return; // dropped by policy
//   inputChannel.send(serializeInput(event));    // existing behavior
// }

// ---- MODIFY: host-side inbound input handling ----
// Where you currently receive input frames on the host, reject when not allowed.
// NATIVE OS EXECUTION REMAINS DISABLED/NO-OP unless the project already has a
// safe executor — this only decides whether to forward to it.
//
// inputChannel.on('input', (frame) => {
//   if (!remoteInputGate.guardInbound()) return; // rejected by policy

```

```
// safeInputExecutor?.execute(frame);          // existing no-op/safe path
// });
```

Part E — Emergency stop

FILE: apps/desktop/src/renderer/src/services/emergencyStop.ts

```
import type { HostPermissionState } from './permissionState';
import { remoteInputGate, applyPermissionsToWebRtc } from './webrtc';
import { sendPermissionSnapshot, type PermissionSnapshotFrame } from './sessionDataChannel';

/**
 * Host emergency stop. Immediately disables remote input, rebroadcasts the
 * permission snapshot (so the viewer stops capturing), and updates local gate
 * state. Does NOT disconnect the WebRTC screen view.
 */
export class EmergencyStop {
  private engaged = false;

  constructor(
    private readonly permissionState: HostPermissionState,
    private readonly send: (frame: PermissionSnapshotFrame) => void,
  ) {}

  isEngaged(): boolean {
    return this.engaged;
  }

  /** Engage: kill input now, broadcast, keep the screen view alive. */
  engage(): void {
    this.engaged = true;
    this.permissionState.setEmergencyStopped(true);
    const permissions = this.permissionState.current();
    applyPermissionsToWebRtc(permissions); // local gate denies immediately
    sendPermissionSnapshot(this.send, permissions); // tell the viewer to stop
  }

  /** Release: clears the stop. Input still requires host acceptance + policy. */
  release(): void {
    this.engaged = false;
    this.permissionState.setEmergencyStopped(false);
    const permissions = this.permissionState.current();
    applyPermissionsToWebRtc(permissions);
    sendPermissionSnapshot(this.send, permissions);
  }
}
```

Part F — UI

FILE: apps/desktop/src/renderer/src/components/PolicyStatusBanner.tsx

```
import React from 'react';
import {
  getPolicyDenialMessage,
```

```

    type DeviceAccessPolicySnapshot,
    type SessionPermissionSet,
  } from '@remotedesk/shared';

/**
 * Shows the current policy + permission status. Read-only; reflects the
 * authoritative permission set (host) or the applied snapshot (viewer).
 */
export function PolicyStatusBanner({
  policy,
  permissions,
  fromCache,
}): {
  policy: DeviceAccessPolicySnapshot | null;
  permissions: SessionPermissionSet | null;
  fromCache?: boolean;
}) {
  const trust = policy?.trustStatus ?? 'unknown';
  const input = permissions?.remoteInput ?? 'disabled';

  const tone =
    trust === 'blocked' || input === 'blocked'
      ? 'danger'
      : input === 'enabled'
        ? 'ok'
        : 'warn';

  return (
    <div className={`policy-banner policy-banner--${tone}`} role="status">
      <span className="policy-banner__trust">Device: {trust}</span>
      <span className="policy-banner__sep"> · </span>
      <span className="policy-banner__input">Remote input: {input}</span>
      {permissions && (
        <>
          <span className="policy-banner__sep"> · </span>
          <span>Clipboard: {permissions.clipboardSync ? 'on' : 'off'}</span>
          <span className="policy-banner__sep"> · </span>
          <span>File transfer: {permissions.fileTransfer ? 'on' : 'off'}</span>
        </>
      )}
      {permissions?.emergencyStopped && (
        <span className="policy-banner__estop"> · EMERGENCY STOP ACTIVE</span>
      )}
      {fromCache && <span className="policy-banner__cache"> · showing last known (offline)</span>}
      {policy === null && (
        <div className="policy-banner__reason">{getPolicyDenialMessage('missing_policy')}</div>
      )}
    </div>
  );
}

```

FILE: apps/desktop/src/renderer/src/components/EmergencyStopButton.tsx

```

import React from 'react';

/**

```

```

* Host-only emergency stop / resume control. Disables remote input instantly
* without disconnecting the screen view.
*/
export function EmergencyStopButton({
  engaged,
  onEngage,
  onRelease,
}): {
  engaged: boolean;
  onEngage: () => void;
  onRelease: () => void;
}) {
  return (
    <button
      type="button"
      className={`estop-btn ${engaged ? 'estop-btn--engaged' : ''}`}
      onClick={engaged ? onRelease : onEngage}
      aria-pressed={engaged}
    >
      {engaged ? 'Resume control' : 'Emergency stop input'}
    </button>
  );
}

```

FILE: apps/desktop/src/renderer/src/components/RemoteSessionView.tsx [MODIFY]

```

// ...existing imports...
import { PolicyStatusBanner } from './PolicyStatusBanner';
import { EmergencyStopButton } from './EmergencyStopButton';
import { applyPermissionsToWebRtc, remoteInputGate } from '../services/webrtc';
import type { DeviceAccessPolicySnapshot, SessionPermissionSet } from '@remotedesk/shared';

// ---- ADD: state for policy + permissions (wire to your existing session state) ----
// const [policy, setPolicy] = useState<DeviceAccessPolicySnapshot | null>(null);
// const [permissions, setPermissions] = useState<SessionPermissionSet | null>(null);
// const isHost = /* your existing role flag */;
// const emergencyStop = /* EmergencyStop instance constructed at session start */;

// Whenever permissions change (host computes, viewer receives over data channel):
//   applyPermissionsToWebRtc(permissions);
//   setPermissions(permissions);

// ---- ADD: render the banner above the video, and gate input UI ----
// <PolicyStatusBanner policy={policy} permissions={permissions} />
//
// {isHost && (
//   <EmergencyStopButton
//     engaged={!permissions?.emergencyStopped}
//     onEngage={() => emergencyStop.engage()}
//     onRelease={() => emergencyStop.release()}
//   />
// )}
//
// VIEWER read-only mode: when input is not allowed, show a read-only overlay and
// do NOT attach input listeners.
// const readOnly = !remoteInputGate.inputAllowed();

```

```
// <div className={readOnly ? 'remote-video remote-video--readonly' : 'remote-video'}>
//   {readOnly && <div className="readonly-overlay">View only — remote input disabled</div>}
//   {/* existing <video> element unchanged so screen viewing keeps working */}
// </div>
//
// IMPORTANT: only attach mouse/keyboard capture handlers when
// remoteInputGate.inputAllowed() is true; detach immediately when it flips false
// (e.g. inside a useEffect that depends on `permissions`).
```

Part G — Tests

FILE: tests/permissions/policyEvaluator.test.ts

```
import { describe, it, expect } from 'vitest';
import {
  evaluateSessionStartPolicy,
  canEnableRemoteInput,
  canEnableClipboard,
  canEnableFileTransfer,
  deriveSessionExpiry,
  isSessionExpired,
  buildSessionPermissionSet,
  type DeviceAccessPolicySnapshot,
} from '@remotedesk/shared';

function policy(over: Partial<DeviceAccessPolicySnapshot> = {}): DeviceAccessPolicySnapshot {
  return {
    deviceId: 'dev-1',
    trustStatus: 'trusted',
    unattendedAccessEnabled: false,
    remoteInputEnabled: true,
    clipboardSyncEnabled: true,
    fileTransferEnabled: true,
    requiresSessionApproval: true,
    maxSessionMinutes: null,
    updatedAt: new Date().toISOString(),
    ...over,
  };
}

describe('policyEvaluator', () => {
  it('blocked device denies session start and all risky features', () => {
    const p = policy({ trustStatus: 'blocked' });
    expect(evaluateSessionStartPolicy(p).allowed).toBe(false);
    expect(canEnableRemoteInput(p, true).reason).toBe('device_blocked');
    expect(canEnableClipboard(p, true).reason).toBe('device_blocked');
    expect(canEnableFileTransfer(p, true).reason).toBe('device_blocked');
  });

  it('untrusted device denies remote input', () => {
    const p = policy({ trustStatus: 'untrusted' });
    expect(canEnableRemoteInput(p, true).reason).toBe('device_untrusted');
  });

  it('missing policy fails closed for every risky feature', () => {
```

```

    expect(evaluateSessionStartPolicy(null).reason).toBe('missing_policy');
    expect(canEnableRemoteInput(null, true).reason).toBe('missing_policy');
    expect(canEnableClipboard(null, true).reason).toBe('missing_policy');
    expect(canEnableFileTransfer(null, true).reason).toBe('missing_policy');
  });

  it('remote input is denied until the host accepts', () => {
    expect(canEnableRemoteInput(policy(), false).reason).toBe('host_not_accepted');
    expect(canEnableRemoteInput(policy(), true).allowed).toBe(true);
  });

  it('host accepted + trusted + enabled allows remote input', () => {
    expect(canEnableRemoteInput(policy(), true).allowed).toBe(true);
  });

  it('emergency stop blocks input even when otherwise allowed', () => {
    expect(canEnableRemoteInput(policy(), true, true).reason).toBe('emergency_stop_active');
  });

  it('feature disabled by policy is denied', () => {
    expect(canEnableRemoteInput(policy({ remoteInputEnabled: false }), true).reason).toBe('feature_disabled_by_policy');
    expect(canEnableClipboard(policy({ clipboardSyncEnabled: false }), true).reason).toBe('feature_disabled_by_policy');
    expect(canEnableFileTransfer(policy({ fileTransferEnabled: false }),
true).reason).toBe('feature_disabled_by_policy');
  });

  it('maxSessionMinutes derives an expiry timestamp', () => {
    const start = new Date('2026-06-15T10:00:00.000Z');
    const expiry = deriveSessionExpiry(policy({ maxSessionMinutes: 30 }), start);
    expect(expiry).toBe('2026-06-15T10:30:00.000Z');
    expect(deriveSessionExpiry(policy({ maxSessionMinutes: null }), start)).toBeNull();
  });

  it('isSessionExpired respects the derived expiry', () => {
    const start = new Date('2026-06-15T10:00:00.000Z');
    const expiry = deriveSessionExpiry(policy({ maxSessionMinutes: 30 }), start);
    expect(isSessionExpired(expiry, new Date('2026-06-15T10:29:59.000Z'))).toBe(false);
    expect(isSessionExpired(expiry, new Date('2026-06-15T10:30:00.000Z'))).toBe(true);
    expect(isSessionExpired(null)).toBe(false);
  });

  it('buildSessionPermissionSet marks blocked vs disabled correctly', () => {
    const blocked = buildSessionPermissionSet({ policy: policy({ trustStatus: 'blocked' }), hostAccepted: true,
emergencyStopped: false, version: 1 });
    expect(blocked.remoteInput).toBe('blocked');

    const notAccepted = buildSessionPermissionSet({ policy: policy(), hostAccepted: false, emergencyStopped: false,
version: 2 });
    expect(notAccepted.remoteInput).toBe('disabled');

    const enabled = buildSessionPermissionSet({ policy: policy(), hostAccepted: true, emergencyStopped: false, version: 3
});
    expect(enabled.remoteInput).toBe('enabled');

    const estopped = buildSessionPermissionSet({ policy: policy(), hostAccepted: true, emergencyStopped: true, version: 4
});
    expect(estopped.remoteInput).toBe('disabled');
  });

```

```
    expect(estopped.emergencyStopped).toBe(true);
  });
});
```

Part H — Reports

FILE: [generated-pack15-desktop-policy-enforcement-merge-summary.md](#)

Pack 15 (Desktop) — Device Policy Enforcement · Merge Summary

What this slice does

Enforces persisted device access policies inside the desktop session flow:

fail-closed evaluation, host-authoritative permission snapshots over the data channel, a bidirectional remote-input gate, and a host emergency stop — without breaking existing WebRTC screen viewing and without adding any native input executor.

Files created (10)

- packages/shared/src/permissions/devicePolicy.ts
- packages/shared/src/permissions/policyEvaluator.ts
- packages/shared/src/permissions/index.ts
- apps/desktop/src/renderer/src/services/devicePolicyClient.ts
- apps/desktop/src/renderer/src/services/permissionState.ts
- apps/desktop/src/renderer/src/services/remoteInputGate.ts
- apps/desktop/src/renderer/src/services/emergencyStop.ts
- apps/desktop/src/renderer/src/components/PolicyStatusBanner.tsx
- apps/desktop/src/renderer/src/components/EmergencyStopButton.tsx
- tests/permissions/policyEvaluator.test.ts

Files modified (4)

- packages/shared/src/index.ts — export permissions barrel.
- apps/desktop/src/renderer/src/services/sessionDataChannel.ts — add permission.snapshot frame + send/handle helpers.
- apps/desktop/src/renderer/src/services/webrtc.ts — gate outbound (viewer) + inbound (host) input on the shared gate.
- apps/desktop/src/renderer/src/components/RemoteSessionView.tsx — banner, emergency stop, viewer read-only mode.

Files intentionally NOT touched

- apps/desktop/src/main/index.ts — no native input executor added.
- apps/desktop/src/preload/index.ts — no new privileged bridge required.
- apps/desktop/src/renderer/src/services/socket.ts — signaling unchanged.
- apps/api/** , apps/web/** — policy already persisted; no runtime changes needed.

Remaining manual wiring risks

1. Confirm the real policy route; if different, change ONLY `DEVICE\_POLICY\_ENDPOINT`.
2. Merge `permission.snapshot` into your actual DataChannelFrame union + dispatch.
3. In RemoteSessionView, attach/detach mouse+keyboard capture based on `remoteInputGate.inputAllowed()` and re-run when permissions change.
4. Construct one `HostPermissionState` + `EmergencyStop` at session start (host) and call `permissionState.setPolicy(...)` after the policy fetch.
5. Keep native OS execution no-op unless a safe executor already exists.

Verify

- `pnpm --filter @remotedesk/shared build && pnpm -r test`
- Manual: blocked/untrusted/missing policy => input shows blocked/disabled; emergency stop flips viewer to read-only instantly while video keeps streaming.

FILE: [generated-pack15-desktop-policy-enforcement-code-review.md](#)

Pack 15 (Desktop) — Code Review Notes

Strengths

- Evaluator is pure + fail-closed: null policy, blocked, untrusted, and missing host acceptance all deny risky features.
- Host is the single source of truth; viewer only consumes versioned snapshots and ignores stale versions (no re-enable after revoke).
- Remote input gated on BOTH ends (viewer outbound + host inbound).
- Emergency stop is instant and does not tear down the screen view.
- Blocked input logged as structured, non-sensitive diagnostics (counts only).
- No native input executor introduced; existing safe/no-op path preserved.

Reviewer checklist

- [] DataChannelFrame union + dispatcher updated for permission.snapshot.
- [] Capture handlers attach/detach on permission changes (useEffect dependency).
- [] Policy route matches DEVICE_POLICY_ENDPOINT.
- [] Session expiry is re-checked on a timer if you want hard cutoff mid-session.
- [] No tokens/policy values written to logs (client logs only reason + ids).

Safety

- Remote input OFF by default; requires trusted device + policy-enabled + host accept.
- Native OS execution remains disabled/no-op.
- Display cache is never used to enable features, only to show last known status.

FILE: [generated-pack15-desktop-policy-enforcement-manifest.json](#)

```
{
  "pack": "pack15-desktop-policy-enforcement",
  "title": "Desktop Device Access Policy Enforcement + Remote Input Safety Gates",
  "actualFileCount": 14,
  "filesCreated": [
    "packages/shared/src/permissions/devicePolicy.ts",
    "packages/shared/src/permissions/policyEvaluator.ts",
    "packages/shared/src/permissions/index.ts",
    "apps/desktop/src/renderer/src/services/devicePolicyClient.ts",
    "apps/desktop/src/renderer/src/services/permissionState.ts",
    "apps/desktop/src/renderer/src/services/remoteInputGate.ts",
    "apps/desktop/src/renderer/src/services/emergencyStop.ts",
    "apps/desktop/src/renderer/src/components/PolicyStatusBanner.tsx",
    "apps/desktop/src/renderer/src/components/EmergencyStopButton.tsx",
    "tests/permissions/policyEvaluator.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/index.ts",
    "apps/desktop/src/renderer/src/services/sessionDataChannel.ts",
    "apps/desktop/src/renderer/src/services/webrtc.ts",
    "apps/desktop/src/renderer/src/components/RemoteSessionView.tsx"
  ],
  "filesNotTouched": [
    "apps/desktop/src/main/index.ts",
    "apps/desktop/src/preload/index.ts",
    "apps/desktop/src/renderer/src/services/socket.ts",
    "apps/api/**",
    "apps/web/**"
  ]
}
```

```

],
"dependenciesRequired": {
  "new": [],
  "existing": ["@remotedesk/shared", "react", "vitest"],
  "assumedFromProject": [
    "existing WebRTC input data channel (send + receive)",
    "existing SessionDataChannel frame union + dispatcher",
    "existing RemoteSessionView role flag (host vs viewer)",
    "device policy persisted + served by the API"
  ]
},
"knownLimitations": [
  "Native OS input execution remains disabled/no-op (no executor added).",
  "Policy route is behind DEVICE_POLICY_ENDPOINT; confirm/adjust to real API.",
  "Session hard-cutoff at expiry requires a timer in the session view if desired.",
  "Display cache is for status only; never enables features."
],
"remainingManualWiringRisks": [
  "Merge permission.snapshot into the real DataChannelFrame union + dispatch.",
  "Attach/detach input capture on permission changes.",
  "Construct HostPermissionState + EmergencyStop at session start and feed policy in."
],
"estimatedCompletionAfterCarefulMerge": "60-66%"
}

```

Desktop Pack 15 done: 14 files (10 created, 4 modified). A pure fail-closed policy evaluator, a host-authoritative permission state that broadcasts versioned snapshots over the data channel, a bidirectional remote-input gate (viewer outbound + host inbound), a host emergency stop that kills input instantly without dropping the screen view, and the UI banner + read-only viewer mode.

The clean bit: remote input is off by default and only flips on for a trusted, policy-enabled device after the host accepts; the viewer never computes its own permissions, it just applies the host's versioned snapshot and ignores stale frames so a reordered packet can't re-enable input after a revoke. Native OS execution stays no-op, and the display cache is never allowed to enable a risky feature. The screen view keeps working through all of it.